

这是一份关于**排序算法 (Sorting Algorithms)** 第一部分的详细总结，基于华南理工大学软件学院 2024 秋季学期的课件。

本讲座主要涵盖了排序的基本概念、术语，以及几类基础的排序算法：冒泡排序、选择排序、插入排序和希尔排序。

排序算法 (Sorting I) - 核心内容总结

1. 简介与术语 (Introduction & Terminology)

什么是排序？

排序是将一组记录 $r_1, r_2, \dots, r_n$ 按照其键值 (key values) $k_1, k_2, \dots, k_n$ 进行重新排列的过程，使得重新排列后的键值满足非递减顺序 ($k_{s1} \leq k_{s2} \leq \dots \leq k_{sn}$)。

关键术语

- 比较排序 (Comparison-based sorting):** 仅通过比较键值大小 ($<$ 或 $>$) 来确定顺序的算法。
- 内部排序 vs 外部排序 (Internal vs External):**
 - 内部排序:** 数据量小，完全在内存 (RAM) 中进行。
 - 外部排序:** 数据量太大，无法一次性装入内存，需要借助磁盘存储。
- 空间复杂度 (Space):**
 - 原地排序 (In-place):** 不需要额外复制数据，仅需 $O(1)$ 的辅助空间。
 - 非原地排序可能需要 $O(n)$ 的辅助空间。
- 稳定性 (Stability):**
 - 定义:** 如果两个记录具有相同的键值，排序算法是否能保留它们在原数组中的相对顺序。
 - 重要性:** 对于多重排序（例如先按名字排，再按国家排）非常重要。
 - 例子:** 原始序列 $r_1(7), r_2(5), r_3(8), r_4(3), r_5(7)$ 。如果排序后 $r_1(7)$ 依然在 $r_5(7)$ 前面，则是稳定的。

效率界限

- 下界:** 至少需要检查每个元素一次，所以最低时间复杂度是 $\Omega(n)$ 。
- 常见复杂度:** 从简单的 $O(n^2)$ 到更快的 $O(n \log n)$ 。

2. $\Theta(n^2)$ 基础排序算法

这一类算法通常通过比较和交换相邻或特定的元素来工作。虽然简单，但在数据量大时效率较低。

冒泡排序 (Bubble Sort)

- 原理:** 重复遍历列表，比较相邻元素 (i 和 $i+1$)。如果顺序错误 ($A[i] > A[i+1]$) 就交换它们。每一轮遍历会将当前未排序部分中最大的元素“冒泡”到其最终位置。
- 代码逻辑:**

```
for (int i=0; i<n-1; i++)  
    for (int j=n-1; j>i; j--) // 从后往前  
        if (Comp::prior(A[j], A[j-1])) swap(A, j, j-1);
```

- 分析:**
 - 时间:** 总是进行 $\Theta(n^2)$ 次比较（无论最好最坏情况）。
 - 交换:** 最坏/平均情况 $\Theta(n^2)$ 次交换，最好情况（已排序）0 次交换。
 - 特性:** 稳定 (Stable)，原地 (In-place)。

选择排序 (Selection Sort)

- 原理:** 在第 i 轮中，扫描剩余未排序的部分，找到最小元素的下标，并将其与位置 i 的元素交换。

• 代码逻辑:

```
for (int i=0; i<n-1; i++) {
    int lowindex = i;
    for (int j=n-1; j>i; j--) // 寻找最小值
        if (Comp::lt(A[j], A[lowindex])) lowindex = j;
    swap(A, i, lowindex); // 放到位置 i
}
```

• 分析:

- 时间: 总是 $\Theta(n^2)$ 次比较, 因为必须扫描剩余部分。
- 交换: 非常少, 总共只需 $n - 1$ 次交换 ($\Theta(n)$)。
- 特性: 不稳定 (Not Stable), 原地 (In-place)。
- 不稳定举例: 3 9 3 2 5 第一轮会将 2 和第一个 3 交换, 导致两个 3 的相对顺序改变。

 插入排序 (Insertion Sort)

- 原理: 构建有序序列, 对于未排序数据, 在已排序序列中从后向前扫描, 找到相应位置并插入。类似于整理扑克牌。
- 代码逻辑:

```
for (int i=1; i<n; i++) // 插入第 i 个记录
    for (int j=i; (j>0) && (Comp::prior(A[j], A[j-1])); j--)
        swap(A, j, j-1); // 通过不断交换将元素移到正确位置
```

• 分析:

- 最好情况 (已排序): $\Theta(n)$, 只需比较不需交换。
- 最坏情况 (逆序): $\Theta(n^2)$ 。
- 平均情况: $\Theta(n^2)$ 。
- 关键点: 插入排序对于几乎有序 (almost sorted) 的数据非常高效。
- 特性: 稳定 (Stable), 原地 (In-place)。

 三种算法对比总结

算法	最好时间复杂度	平均时间复杂度	最坏时间复杂度	交换次数 (平均/最坏)	稳定性
冒泡排序	$\Theta(n^2)$ (比较)	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	✅ 稳定
选择排序	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$ (最少)	❌ 不稳定
插入排序	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	✅ 稳定

逆序对 (Inversion): 任何一对 (i, j) 满足 $i < j$ 但 $A[i] > A[j]$ 。任何仅交换相邻元素的算法 (如冒泡和插入排序) 平均都需要 $\Omega(n^2)$ 的时间, 因为平均逆序对数量是 $n(n - 1)/4$ 。要突破 $O(n^2)$, 必须交换**非相邻**元素。

3. 希尔排序 (Shellsort) - 突破 $O(n^2)$

希尔排序是插入排序的改进版, 也称为**缩小增量排序 (diminishing increment sort)**。它的核心思想是通过比较和交换**不相邻**的元素, 先让数组变得“宏观上有序”, 最后再进行一次普通的插入排序。

工作原理

1. 分治思想: 将数组按间隔 (增量) h_k 分成若干子序列。
2. 子排序: 对每个子序列进行独立的插入排序。此时相距 h_k 的元素是有序的。
3. 缩小增量: 减小 h_k , 重复上述过程, 直到 $h_1 = 1$ (即标准的插入排序)。

增量序列 (Increment Sequence)

希尔排序的效率高度依赖于增量序列的选择:

- Shell 原始增量: $N/2, N/4, \dots, 1$ 。

- 最坏情况: $\Theta(n^2)$ (因为奇数位置和偶数位置直到最后才比较)。
- Hibbard 增量:** $1, 3, 7, \dots, 2^k - 1$ (相邻增量互质)。
 - 最坏情况: $\Theta(n^{3/2})$ 。
- Sedgewick 增量:** $1, 5, 19, 41, \dots$ (复杂的公式)。
 - 最坏情况: $O(n^{4/3})$, 平均情况 $O(n^{7/6})$ 。这是目前实践中已知的最好序列。

特性

- 原地 (In-place)**。
- 不稳定 (Not Stable)**。
 - 例子: $3\ 5\ 10\ 8\ 7\ 2\ 8\ 1\ 20\ 6$, 使用增量 2 排序后, 两个 8 的相对顺序可能会改变。

这是一份关于 $O(N \log N)$ 排序算法 (Sorting II) 的详细总结, 基于华南理工大学软件学院 2024 秋季学期的课件。

本讲座重点介绍了三种高效的排序算法: **堆排序 (Heap Sort)**、**归并排序 (Merge Sort)** 和 **快速排序 (Quick Sort)**。

排序算法进阶 (Sorting II) - $O(N \log N)$ 算法

1. 堆排序 (Heap Sort)

堆排序利用了**最大堆 (Max-Heap)**的特性来进行排序。它的核心思想是利用堆的数据结构来管理数据, 从而无需额外的存储空间 (In-place)。

核心原理

- 构建堆:** 首先将数组构建为一个最大堆。此时, 数组的第一个元素 $A[1]$ (根节点) 是最大值。
- 删除最大值 (DeleteMax):**
 - 将堆顶的最大元素取出。
 - 由于堆的大小减少了 1, 数组末尾空出了一个位置。
 - 我们将取出的最大元素直接存放到数组的这个末尾位置。
- 重复:** 重复执行 DeleteMax N 次。数据会按照从大到小的顺序依次从堆中弹出, 并填充到数组的末尾 (从后往前填)。最终, 数组就变成了有序的 (从小到大)。

性能分析

- 时间复杂度:**
 - 构建堆: $O(N)$ 。
 - 删除 N 个元素: 每次删除耗时 $O(\log N)$, 总共 $O(N \log N)$ 。
 - 总时间:** 最好、平均和最坏情况均为 $O(N \log N)$ 。
- 空间复杂度:** **原地排序 (In-place)**, 不需要额外辅助数组。
- 稳定性:** **不稳定 (Not Stable)**。

2. 分治策略 (Divide and Conquer)

归并排序和快速排序都使用了**分治策略**。其基本步骤是:

- 分 (Divide):** 将问题分解为更小的部分。
- 治 (Conquer):** 独立解决这些小部分 (通常是递归地)。
- 合 (Combine):** 合并这些部分的解以得到整体解。

3. 归并排序 (Merge Sort)

归并排序侧重于“合”的过程。

算法流程

- 分:** 将数组从中间切分为两半。
- 治:** 递归地对左半部分和右半部分进行排序。
- 合 (Merge):** 将两个已经排序的半部分合并成一个完整的有序数组。

合并操作 (Merging)

- 需要一个**辅助数组 (Auxiliary Array)**。
- 逻辑:** 比较两个子数组的头部元素，将较小的那个放入辅助数组，并移动对应的指针。重复此过程直到某一半为空，然后将另一半剩余的元素全部复制过去。
- 最后将辅助数组的内容拷回原数组。

性能分析

- 递推公式:** $T(N) = 2T(N/2) + O(N)$ 。
- 时间复杂度:** $O(N \log N)$ 。
- 空间复杂度:** $O(N)$ 。因为它需要一个与原数组等大的辅助数组，所以它**不是**原地排序。
- 稳定性:** **稳定 (Stable)**。在合并时，如果遇到相等的值，优先取左边的元素即可保证稳定性。

4. 快速排序 (Quick Sort)

快速排序是实践中最快的已知排序算法之一，它侧重于“分”的过程。

算法流程

- 选基准 (Pick Pivot):** 从数组中选择一个元素作为基准值 (Pivot)。
- 分区 (Partition):** 将数组重排，使得所有**小于**基准的元素放在左边 (S_1)，所有**大于**基准的元素放在右边 (S_2)。
- 递归:** 对 S_1 和 S_2 递归地进行快速排序。

关键细节

- 基准的选择 (Pivot Selection):**
 - 选第一个元素:** 如果输入是预排序的，这是极差的选择，会导致 $O(N^2)$ 的复杂度。
 - 随机选取:** 安全但生成随机数开销大。
 - 三数中值法 (Median-of-Three):** 选取左端、右端和中间位置的三个数，取它们的中位数作为基准。这是推荐的做法。
- 分区策略 (Partitioning Strategy):**
 - 使用两个指针 i (从左向右) 和 j (从右向左)。
 - i 停在大于基准的元素上， j 停在小于基准的元素上，然后交换它们。
 - 当 i 和 j 相遇 (交错) 时停止，最后将基准放到正确的位置。
- 小数组优化 (Cutoff):**
 - 对于很小的数组 (如 $N \leq 10$)，快速排序的递归开销较大。
 - 优化:** 当子数组很小时，改用**插入排序 (Insertion Sort)**。

性能分析

- 平均时间复杂度:** $O(N \log N)$ 。
- 最坏时间复杂度:** $O(N^2)$ (例如总是选到最小或最大元素作为基准，导致一边为空)。但通过“三数中值法”可以有效避免。
- 空间复杂度:** $O(\log N)$ (递归栈的空间)，被认为是**原地排序 (In-place)**。
- 稳定性:** **不稳定 (Not Stable)**，因为存在长距离的交换。

5. 三种算法对比总结

特性	堆排序 (Heap Sort)	归并排序 (Merge Sort)	快速排序 (Quick Sort)
平均时间	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$
最坏时间	$O(N \log N)$	$O(N \log N)$	$O(N^2)$
空间复杂度	$O(1)$ (In-place)	$O(N)$ (需辅助数组)	$O(\log N)$ (递归栈)
稳定性	不稳定 ❌	稳定 ✅	不稳定 ❌
特点	无需额外空间, 性能稳定	稳定, 但需要额外内存	平均最快, 利用缓存好

结论: 快速排序通常是内存中排序的最佳选择, 因为它具有极少的比较次数、较小的内存占用以及良好的局部性 (Cache Locality)。

这是一份关于**线性时间排序 (Linear-Time Sorts)** 和 **外部排序 (External Sorting)** 的详细总结, 基于华南理工大学软件工程学院的课件。

本讲座主要介绍了突破比较排序 $O(N \log N)$ 下界的三种算法: 桶排序、计数排序和基数排序, 以及处理超大规模数据的外部排序技术。

排序算法：突破界限 (Sorting III)

1. 线性时间排序 ("Linear-Time" Sorts)

前面介绍的比较排序算法 (如快速排序、归并排序) 的**时间复杂度下界**是 $O(N \log N)$ 。线性时间排序算法通过**不依赖比较**, 而是利用键值的特定性质来直接计算或分配元素位置, 从而实现 $O(N)$ 的时间复杂度。

1.1 桶排序 (Bucket Sort)

桶排序是一种最简单的线性时间排序, 但对输入数据有严格限制。

- 适用场景:** 键值是 0 到 $N - 1$ 之间数字的排列 (Permutation), 且**没有重复值**。
- 原理:** 直接使用键值作为目标数组的**索引 (Index)**, 将键值 $A[i]$ 放到目标数组 $B[A[i]]$ 的位置。
- 时间复杂度:** $\Theta(N)$ (线性时间)。

改进版桶排序

为了处理更一般的场景, 例如允许**重复键值**或键值范围**大于 N** :

- 方法:** 创建一个由**链表 (Linked List)** 组成的数组 $B[]$ (即桶/箱)。
- 分配:** 将具有相同键值的所有记录放入同一个桶 $B[key]$ 中。
- 收集:** 按照桶的索引顺序, 将所有桶中的链表元素依次取出, 放回原数组。
- 时间复杂度:** 假设键值范围是 M , 总成本为 $\Theta(N + M)$ 。当 $M \approx N$ 时, 复杂度接近 $\Theta(N)$ 。

1.2 计数排序 (Counting Sort)

计数排序是桶排序的一种更正式、更高效的版本, 它消除了链表的开销, 适用于键值范围 M 较小的情况。

- 适用场景:** 键值必须是整数, 且键值范围 $M = \text{MaxKeyValue} - 1$ 不宜太大。
- 原理:**
 - 计数 (Count):** 创建一个大小为 M 的计数数组 $C[]$ 。遍历输入数组 A , 统计每个键值出现的次数, 存入 C 。
 - 累积 (Accumulate):** 修改 C , 使 $C[i]$ 存储**小于或等于**键值 i 的元素总数。这确定了每个键值在最终排序数组中的**结束位置**。
 - 放置 (Place):** 倒序遍历输入数组 A , 将元素 $A[j]$ 放入目标数组 B 的位置 $C[A[j]]$, 然后将 $C[A[j]]$ 减 1。**倒序遍历是为了保证稳定性**。
- 时间复杂度:** $\Theta(N + M)$ 。

1.3 基数排序 (Radix Sort)

基数排序是一种多趟排序算法, 通过依次对键的每个**位 (digit)** 进行稳定排序, 最终实现整体有序。

- **适用场景:** 适用于键值可以分解为多个位（例如数字的十进制位，或字符串的字符）的情况。
- **核心原理 (LSD Radix Sort - 最低位优先):**
 - i. 从最低有效位（Least Significant Digit, LSD）开始。
 - ii. 对所有元素，基于当前位的值，使用**稳定的排序算法**（例如**计数排序**）进行一次排序。
 - iii. 重复此过程，直到最高有效位。
- **稳定性要求:** 每一趟排序必须是稳定的，以确保低位排序的结果不会被高位排序破坏。

性能分析

- 设键值有 r 个位 (passes)，每个位使用 b 进制表示 (base b)。
- 每趟排序（如使用计数排序）的成本是 $\Theta(N + b)$ 。
- **总时间复杂度:** $\Theta(r(N + b))$ 。
- **优化:** 为了最小化时间，需要找到 r 和 b 之间的最佳平衡。例如，对于 N 个不同的键，至少需要 $r = \lceil \log_b N \rceil$ 位。当 r 和 b 取得合适的值时，基数排序可以达到 $O(N)$ 的渐近性能。

2. 外部排序 (External Sorting)

当数据量太大，无法一次性装入主内存 (RAM) 时，就需要使用外部排序，它通过优化磁盘 I/O（输入/输出）操作来进行排序。

2.1 外部排序的挑战

- **I/O 瓶颈:** 磁盘 I/O 操作比内存操作慢得多，因此目标是**最小化磁盘访问次数**。
- **操作单位:** 数据以**块 (Block)** 为单位在磁盘和内存之间传输。

2.2 两趟归并排序 (Two-Pass Merge Sort)

外部排序通常采用归并排序的变体，因为它能有效利用顺序 I/O。

阶段 1: 运行的创建 (Creating Runs)

1. 将大文件读取到内存，每次读取**内存能容纳的最大块**。
2. 对内存中的数据块进行**内部排序**（例如使用快速排序）。
3. 将排好序的数据块（称为一个 **Run**）写回磁盘。
4. 重复此过程直到整个文件被分割成 M 个有序的 Run。

阶段 2: 多路归并 (Multi-way Merging)

- 如果文件被分为 M 个 Run，内存中有 B 个缓冲区 (Block)，我们每次可以合并 k 个 Run，其中 $k \leq B - 1$ 。
- **多路归并:** 同时从 k 个 Run 中读取一个块到内存，找到这 k 个块中的最小元素，将其写入输出缓冲区。重复直到所有 Run 合并为一个。
- **归并趟数:** 如果 M 是 Run 的数量，每次归并 k 个 Run，则总归并趟数 $P = \lceil \log_k M \rceil$ 。

2.3 内存分配 (Memory Allocation)

- 如果内存大小是 M_{RAM} ，Run 的大小就是 M_{RAM} 。
- 将 M_{RAM} 块分配给 k 个输入缓冲区和 1 个输出缓冲区，即 $k + 1 = M_{RAM} / BlockSize$ 。

2.4 优化：替代选择排序 (Replacement Selection)

在创建 Run 阶段，可以使用**替代选择 (Replacement Selection)** 来生成比内存容量**更长**的 Run，从而减少 Run 的总数量 M 。

- **方法:** 使用一个**最小堆**来管理内存中的记录。
 - i. 从输入文件中读取记录填满内存，并构建一个最小堆。
 - ii. 输出堆顶（最小）元素到输出缓冲区。
 - iii. 从输入文件读取下一个元素 X 。
 - iv. 如果 X 大于等于刚输出的元素，则将 X 插入堆中。
 - v. 如果 X 小于刚输出的元素，则 X 属于**下一个 Run**。将 X 存放在一个**后备空间**（不参与当前堆），直到当前 Run 耗尽。
- **优点:** 平均来说，可以生成**两倍于内存大小**的 Run。

3. 排序算法总结对比 (Total Summary)

算法类别	算法	平均时间复杂度	最坏时间复杂度	额外空间	稳定性
$O(N^2)$	插入排序	$O(N^2)$	$O(N^2)$	$O(1)$	稳定
	选择排序	$O(N^2)$	$O(N^2)$	$O(1)$	不稳定
$O(N \log N)$	快速排序	$O(N \log N)$	$O(N^2)$	$O(\log N)$	不稳定
	归并排序	$O(N \log N)$	$O(N \log N)$	$O(N)$	稳定
	堆排序	$O(N \log N)$	$O(N \log N)$	$O(1)$	不稳定
线性时间	计数排序	$\Theta(N + M)$	$\Theta(N + M)$	$\Theta(N + M)$	稳定
	基数排序	$\Theta(r(N + b))$	$\Theta(r(N + b))$	$\Theta(N + b)$	稳定
这份 PPT 详细介绍了 比较排序 (Comparison-based Sorting) 的理论下界，即证明了基于比较的算法不可能比 $O(N \log N)$ 更快，并引入了用于证明下界的**决策树 (Decision Tree) **模型。					

以下是详细的总结：

排序算法：理论下界与决策树 (Sorting IV)

1. 比较排序的渐近下界 (Lower Bound for Comparison Sorting)

速度限制

对于任何基于**比较**（如 $A[i] < A[j]$ ）来确定元素顺序的排序算法，其最坏情况下的时间复杂度不可能低于 $\Omega(N \log N)$ 。

- 已知事实:
 - 排序问题的最低理论时间复杂度是 $\Omega(N)$ （因为至少要读取所有 N 个元素一次）。
 - 目前已知最快的通用比较排序算法（如归并排序、堆排序、快速排序的平均情况）的时间复杂度是 $O(N \log N)$ 。
- 结论: $O(N \log N)$ 算法在渐近意义上是**最优**的。

2. 决策树模型 (The Decision Tree Model)

为了证明这个下界，我们使用**决策树**来抽象任何比较排序算法的执行过程。

决策树的定义

- 模型**: 任何基于比较的排序算法都可以被建模为一个二叉决策树。
- 内部节点 (Internal Nodes)**: 代表一次**二元比较**（例如 $A[i]$ 是否小于 $A[j]$ ）。比较的结果（是/否）决定了算法接下来沿着左分支或右分支继续执行。
- 叶节点 (Leaf Nodes)**: 代表算法执行结束，此时数组中的元素已经排好序。每个叶节点对应于输入序列可能的一种**排列 (Permutation)**。

算法时间复杂度与决策树的关系

- 比较次数:** 排序算法所需要的比较次数等于决策树中从根节点到对应叶节点的路径长度。
- 最坏情况时间复杂度:** 等于决策树的**高度** h (即最长路径的长度)。

3. 排序下界的证明 (Proof of the Sorting Lower Bound)

证明思路

证明的关键在于, 对于 N 个元素的输入, 决策树必须至少有 $N!$ 个叶节点。

- 排列总数 (Total Permutations):** 包含 N 个不同元素的数组, 共有 $N!$ 种可能的初始排列。算法必须为这 $N!$ 种排列中的每一种找到正确的有序序列。
- 叶节点要求:** 因此, 决策树中必须至少有 $N!$ 个叶节点, 每个叶节点对应一种最终的有序排列。
- 树高与叶节点的关系:** 对于一棵高度为 h 的二叉树, 它最多可以有 2^h 个叶节点。
- 推导下界:** 结合以上两点, 必须满足 $2^h \geq N!$ 。
- 计算 h :** 对两边取 $\log_2$, 得到 $h \geq \log_2(N!)$ 。
- Stirling 近似:** 使用斯特林近似 (Stirling's Approximation) 或其他方法对 $\log_2(N!)$ 进行简化, 可以得出:

$$h \geq \log_2(N!) = \Omega(N \log N)$$

结论: 排序算法的最坏情况运行时间 $T(N)$ 必须满足 $T(N) \geq h = \Omega(N \log N)$ 。因此, 任何基于比较的排序算法, 其时间下界是 $\Omega(N \log N)$ 。

4. 选择问题及其下界 (Lower Bounds for Selection Problems)

决策树不仅可以用于排序, 还可以用于分析其他基于比较的**选择问题 (Selection Problems)**, 例如:

- 找到 N 个元素中的**最小元素 (Smallest Item)**。
- 找到 N 个元素中的**第 k 小元素 (k -th Smallest Item)**。
- 找到**中位数 (Median)**。

最小元素查找的下界

问题: 找到 N 个元素中的最小元素, 至少需要多少次比较?

- 证明思路:** 最小元素必须与所有其他 $N - 1$ 个元素进行过比较, 并且在这些比较中都获胜 (即是最小的)。
- 结论:** 任何基于比较的算法要找到最小元素, 至少需要 $N - 1$ 次比较。
 - 决策树视角:** 决策树必须有至少 2^{N-1} 个叶节点, 因此树高 $h \geq N - 1$ 。

总结

对于基于比较的算法:

- 排序 (Sorting):** 下界为 $\Omega(N \log N)$ 。
- 查找最小元素 (FindMin):** 下界为 $\Omega(N)$ (精确地说是 $N - 1$)。



外部排序 (Sorting V)

好的, 这个PPT的内容主要是关于**外部排序 (External Sorting)**, 特别是**外部归并排序 (External Mergesort)**, 以及提高其效率的两种主要方法:
多路归并 (Multiway Merge) 和 **置换选择 (Replacement Selection)** 算法用于创建更长的初始运行段。

以下是内容的详细总结和解释:

外部排序 (External Sorting)

外部排序算法主要用于处理输入规模过大，无法完全放入计算机内部存储器（内存）的情况。

目标与过程

- 主要目标：** 最小化信息从大容量存储设备（如磁盘或磁带）读写到内存的次数，因为I/O操作通常是外部排序的瓶颈。
- 基本过程：** 外部排序通过重复以下步骤来完成：
 - 从磁盘读取部分记录到内存。
 - 在内存中进行一些重新排列（排序/归并）。
 - 将记录写回大容量存储设备。
 - 重复此过程直到文件完全排序。

外部排序模型

- 记录通常存储在磁带上，磁带只能进行顺序访问。
- 假设算法至少在三条磁带上执行。

简单的外部归并排序 (Simple External Mergesort)

这是一种基于两路归并（Two-Way Merge）的外部排序算法。

算法假设

- 使用四条磁带： $T_{a1}, T_{a2}, T_{b1}, T_{b2}$ 。
- 初始数据位于 T_{a1} 。
- 内部存储器（内存）一次可以容纳和排序 M 条记录（在示例中 $M = 3$ ）。

算法步骤 (以 $M = 3$ 为例)

1. 初始运行段构建 (Initial Run-Constructing Pass)

- 从 T_{a1} 连续读取 M 记录到内存，进行内部排序，然后将排好序的 M 条记录作为一个运行段（Run）写回到 T_{b1} 和 T_{b2} 上，进行交替分配。
- 结果：得到一系列长度为 M 的有序运行段，交替存储在 T_{b1} 和 T_{b2} 上。

2. 归并过程 (Merging Passes)

- 第 2 次归并 (2nd Pass):** 以 T_{b1}, T_{b2} 为输入，进行两路归并，将每条磁带上的两个 M 长度的运行段归并成一个 $2M$ 长度的新运行段，交替写到 T_{a1}, T_{a2} 作为输出。
- 第 3 次归并 (3rd Pass):** 以 T_{a1}, T_{a2} 为输入，进行两路归并，将两个 $2M$ 长度的运行段归并成一个 $4M$ 长度的新运行段，交替写回 T_{b1}, T_{b2} 。
- 最终归并 (Final Pass):** 重复归并过程，直到只剩下一个包含所有 N 条记录的运行段。

归并趟数 (Number of Passes)

- 总趟数：** 初始运行段构建趟数 + 归并趟数。
- 归并趟数 $= \log_2(N/M)$ ，其中 N 是总记录数， M 是内存可容纳的记录数。
- 例如：如果初始运行段有 320 个，则需要 $\log_2(320) \approx 9$ 趟归并。

外部归并排序的改进

为了提高效率，可以采取两种主要改进措施：

- 构建尽可能长的初始运行段。

2. 在每次归并时**增加同时归并的运行段数量**。

1. 多路归并 (Multiway Merge)

多路归并将基本的两路归并扩展到 k 路归并。

- **工作原理：** k 路归并与两路归并类似，但它需要从 k 个输入运行段中找到**最小的元素**。
- **实现：** 通常使用**优先队列 (Priority Queue)** 来高效地找出这 k 个元素中的最小值。
- **磁带要求：** k 路归并至少需要 $k + 1$ 条磁带。例如，3-路归并的例子中使用了 T_{a1}, T_{a2}, T_{a3} 作为输入， T_{b1}, T_{b2}, T_{b3} 作为输出。
 - **归并趟数：** 使用 k 路归并后，归并趟数减少为 $\log_k(N/M)$ 。
- **例如：** 320 个初始运行段使用 **5 路归并**，只需 $\log_5(320) \approx 4$ 趟归并，相比两路归并的 9 趟大大减少。

2. 置换选择 (Replacement Selection)

置换选择是一种创建**比内存容量 M 更长**的初始运行段的算法。简单的外部归并排序只能创建长度为 M 的运行段。

- **平均运行段长度：** 置换选择算法平均能创建长度为 $2M$ 的运行段。
- **基于堆排序 (Heapsort)：** 置换选择是堆排序的一种变体，使用了**最小堆 (Min-Heap)**。
 - **主要步骤：**
 - i. 读取 M 条记录到内存，构建最小堆。
 - ii. **重复**直到堆为空：
 - 从堆中删除最小值 (**deleteMin**) 并写入输出磁带。
 - 从输入磁带读取下一条记录 R 。
 - **判断：** 如果 R 的值**大于**刚输出的记录，则将 R 放入堆的根部，并重新排序堆。
 - **否则：** R 属于下一个运行段。将堆中最后一个元素移到根部，用 R 替换最后一个元素，并缩减堆的大小（通过 *LAST* 指针），然后重新排序堆。
 - iii. 当无法再将记录放入当前运行段时，开始构建下一个运行段（“重建堆”）。